FTC DECODE: Making our bot a dog

Design Document

May 3, 2026

Contents

1 Approach **1**
- 1.1 Problem Statement . . . 1
- 1.2 Core Design Decisions . . . 1

2 Pipeline and Math **2**
- 2.1 Coordinate Frames . . . 2
- 2.2 Transforms . . . 2
- 2.3 Pixel → Field Projection . . . 3
- 2.4 Measurement Covariance in Field Coordinates . . . 3
- 2.5 Gating . . . 4
- 2.6 Kalman Filter Per Track . . . 4
- 2.7 Data Association . . . 5
- 2.8 Time Synchronization . . . 5
- 2.9 Top-Level Tracker Tick . . . 6
- 2.10 Key Failure Modes to Design Against . . . 6

1 Approach

1.1 Problem Statement

The robot carries a global-shutter camera and existing ball-detection software that outputs pixel coordinates of detected balls per frame. Robot pose (x_r, y_r, θ_r) in field coordinates is available from odometry. The goal is to convert per-frame pixel detections into stable, persistent ball tracks in field coordinates, fuse detections across time, reject false positives (balls on the ramp, detections outside the field, detections on the robot itself), and expose a target ball $(x_{\text{ball}}, y_{\text{ball}})$ to a chase controller that drives the robot toward it.

1.2 Core Design Decisions

1. **Filter in field coordinates, not image space.** A ball at a fixed field location appears at different pixel coordinates as the robot moves. Tracking in pixel space would make every frame look like the ball is moving, which destroys the benefit of a motion model. Tracking in field

coordinates means a stationary ball has a stationary state and a rolling ball has a constant-velocity state — clean and correct.

2. **Project each pixel detection to a field point by intersecting its back-projected ray with a horizontal plane.** The plane sits at height $h = r_{\text{ball}}$ above the floor (ball center height). This gives a closed-form projection with no depth sensor required.

3. **Kalman filter per track.** State is $[x, y, v_x, v_y]^\top$ in field meters. Balls are either stationary or rolling with slowly-changing velocity, so a constant-velocity model with tunable process noise is appropriate. Balls moving in and out of frame are handled by the standard predict-only-when-unmeasured behavior: covariance grows during occlusion, shrinks again on re-acquisition.

4. **Propagate pixel uncertainty through the projection into a field-coordinate measurement covariance.** Detections far from the camera have much larger range uncertainty than cross-range uncertainty. Treating measurement noise as a scalar destroys association quality at range. We compute a 2×2 covariance per detection via the Jacobian of the unprojection.

5. **Gate detections before association.** Any projected detection that lands outside the field, inside the ramp exclusion polygon, or inside the robot's own footprint is discarded. This prevents balls on the ramp (which violate the flat-floor assumption) and robot-intake self-detections from corrupting tracks.

6. **Mahalanobis-gated greedy association.** For the number of balls on an FTC field, greedy assignment on Mahalanobis distance is sufficient. Hungarian assignment is a drop-in upgrade if needed.

7. **Time-synchronized pose.** The pose used to project a frame's detections must correspond to when the frame was captured, not when it arrived at the tracker. We maintain a ring buffer of pose samples and interpolate to the frame timestamp. Global shutter guarantees one unambiguous capture time per frame.

2 Pipeline and Math

2.1 Coordinate Frames

Three frames, locked in from the start:

- **Field frame F:** origin at one corner of the field; x, y in the floor plane, z up. Units: meters. Heading θ measured counterclockwise from $+x$.
- **Robot frame R:** origin at robot center (matching odometry convention); x forward, y left, z up.
- **Camera frame C:** OpenCV convention — x right, y down, z forward along the optical axis.

2.2 Transforms

All transforms are 4×4 homogeneous matrices.

Camera in robot frame (fixed, from mount geometry):

$$T_{RC} = \begin{bmatrix} R_{RC} & \mathbf{t}_{RC} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{t}_{RC} = (c_x^r, c_y^r, c_z^r)^\top \tag{1}$$

where R_{RC} is built from the camera's mount pitch, yaw, and roll in the robot frame and the OpenCV-to-robot axis convention change.

Robot in field frame (time-varying, from odometry):

$$T_{FR}(t) = \begin{bmatrix} R_z(\theta_r(t)) & (x_r(t), y_r(t), 0)^\top \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2}$$

Camera in field frame:

$$T_{FC}(t) = T_{FR}(t) \cdot T_{RC} \tag{3}$$

From $T_{FC}(t)$, extract:

- Camera optical center: $\mathbf{o}_F(t) = T_{FC}(t) \cdot (0, 0, 0, 1)^\top$
- Rotation field-from-camera: $R_{FC}(t)$

2.3 Pixel → Field Projection

Given camera intrinsics K and distortion coefficients $\mathbf{d}$, for each detection (u, v):

Step 1 — Undistort the pixel using `cv2.undistortPoints`$(u, v, K, \mathbf{d})$ to get $(u_{\text{ud}}, v_{\text{ud}})$.

Step 2 — Back-project to a camera-frame ray:

$$x_n = \frac{u_{\text{ud}} - c_x}{f_x}, \quad y_n = \frac{v_{\text{ud}} - c_y}{f_y}, \quad \mathbf{d}_C = (x_n, y_n, 1)^\top \tag{4}$$

Step 3 — Rotate into field frame:

$$\mathbf{d}_F = R_{FC} \cdot \mathbf{d}_C \tag{5}$$

Step 4 — Intersect with horizontal plane $z = h$ where $h = r_{\text{ball}}$:

$$s = \frac{h - o_{F,z}}{d_{F,z}} \tag{6}$$

Reject if $s \leq 0$ (ray points up or ball would be behind camera) or $s > s_{\max}$ (beyond plausible field distance).

Step 5 — Field hit point:

$$\mathbf{p}_F = \mathbf{o}_F + s \cdot \mathbf{d}_F \tag{7}$$

The measurement is $\mathbf{z} = (p_{F,x}, p_{F,y})^\top$.

2.4 Measurement Covariance in Field Coordinates

Let σ_{px} be the pixel-space standard deviation of detection error (start at 1.5 px, measure empirically later). Pixel covariance:

$$\Sigma_{\text{px}} = \sigma_{\text{px}}^2 \cdot I_2 \tag{8}$$

The projection $(u, v) \mapsto (p_{F,x}, p_{F,y})$ is nonlinear. Linearize with the Jacobian

$$J = \frac{\partial(p_{F,x}, p_{F,y})}{\partial(u, v)} \tag{9}$$

computed numerically by finite differences ($\delta = 0.5$ px on each axis). The field-frame measurement covariance is

$$R_{\text{meas}} = J \cdot \Sigma_{\text{px}} \cdot J^{\top} \in \mathbb{R}^{2\times 2} \tag{10}$$

This covariance is correctly elongated along the viewing ray (range uncertainty much greater than cross-range) and grows with distance from the camera. Heuristically, lateral uncertainty scales as $\sigma_{\text{px}} \cdot d/f_x$ and range uncertainty as $\sigma_{\text{px}} \cdot d^2/(f_x \cdot h_{\text{cam}})$, where h_{cam} is camera height above the ball plane.

2.5 Gating

All gates applied to each projected, field-coordinate detection **before** association.

Field bounds: reject if (p_x, p_y) lies outside the field rectangle, with a 10 cm inward margin.

Ramp exclusion: reject if (p_x, p_y) lies inside the ramp polygon, expanded outward by $\sim$20 cm on the side the camera typically views from. This compensates for the flat-floor assumption failing on the ramp (a ball on the ramp projects to a point beyond its true location).

Robot self-footprint: transform the robot's footprint polygon from robot frame to field frame using the current pose. Reject detections inside it.

Range: already enforced inside projection ($0 < s \leq s_{\max}$).

2.6 Kalman Filter Per Track

State vector:

$$\mathbf{x} = (p_x, p_y, v_x, v_y)^{\top} \tag{11}$$

Constant-velocity motion model with variable Δt:

$$F(\Delta t) = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{12}$$

Process noise (white acceleration, variance σ_a^2):

$$Q(\Delta t) = \sigma_a^2 \begin{bmatrix} \Delta t^4/4 & 0 & \Delta t^3/2 & 0 \\ 0 & \Delta t^4/4 & 0 & \Delta t^3/2 \\ \Delta t^3/2 & 0 & \Delta t^2 & 0 \\ 0 & \Delta t^3/2 & 0 & \Delta t^2 \end{bmatrix} \tag{13}$$

Start with $\sigma_a = 3.0$ m/s^2.

Measurement model:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad R = R_{\text{meas}} \text{ (per-detection, from 2.4)} \tag{14}$$

Predict (time-update from t_{last} to t, $\Delta t = t - t_{\text{last}}$):

$$\hat{\mathbf{x}} = F\mathbf{x} \tag{15}$$

$$\hat{P} = FPF^\top + Q \tag{16}$$

Update with measurement $\mathbf{z}$:

$$\mathbf{y} = \mathbf{z} - H\hat{\mathbf{x}} \quad \text{(innovation)} \tag{17}$$

$$S = H\hat{P}H^\top + R \quad \text{(innovation covariance)} \tag{18}$$

$$K = \hat{P}H^\top S^{-1} \quad \text{(Kalman gain)} \tag{19}$$

$$\mathbf{x} = \hat{\mathbf{x}} + K\mathbf{y} \tag{20}$$

$$P = (I - KH)\hat{P} \tag{21}$$

2.7 Data Association

For each frame, after predicting all tracks forward to the current timestamp:

1. For every (track i, detection j) pair, compute Mahalanobis distance:

$$d_{ij}^2 = (\mathbf{z}_j - H\hat{\mathbf{x}}_i)^\top S_i^{-1} (\mathbf{z}_j - H\hat{\mathbf{x}}_i) \tag{22}$$

2. **Gate:** discard pairs with $d_{ij}^2 > \chi^2_{\text{thresh}} = 9.21$ (2 DOF, 99% confidence).
3. **Greedy assign:** sort surviving pairs by d_{ij}^2; iterate and assign each pair if both track and detection are still unclaimed.
4. **Unmatched detections** $\rightarrow$ spawn new tracks. Initial state $(z_x, z_y, 0, 0)^\top$; initial P with position block from R_{meas} and large velocity variance ($1.0\,\text{m}^2/\text{s}^2$).
5. **Unmatched tracks** $\rightarrow$ coast (no update); increment `frames_since_seen`.
6. **Delete** tracks with `frames_since_seen` > 15, or where position covariance trace exceeds a threshold.

2.8 Time Synchronization

The pose timestamp must match the frame capture timestamp, not frame arrival. Mechanism:

- RC streams $(t_{\text{RC}}, x, y, \theta)$ over UDP at 50 Hz.
- Pi keeps a ring buffer (capacity $\sim$500 samples) of received pose messages.
- At startup, Pi and RC perform a 10-message ping-pong clock-sync handshake, estimating the clock offset assuming symmetric latency. All incoming RC timestamps are corrected to Pi monotonic time. The offset standard deviation is logged; warn if >5 ms.
- For each frame at Pi time t, retrieve pose by linear interpolation between bracketing buffer samples. Heading θ interpolated with wrap-aware unwrapping.

2.9 Top-Level Tracker Tick

For each camera frame at time t with detections $\{(u_j, v_j)\}$:

1. Fetch pose (x_r, y_r, θ_r) from the buffer at time t.
2. Build $T_{FC}(t) = T_{FR}(t) \cdot T_{RC}$.
3. For each pixel detection: project to (p_x, p_y) with covariance R_{meas}; drop failures.
4. Apply gating: field bounds, ramp, robot footprint. Keep survivors.
5. Predict each existing track forward to time t.
6. Run association: matched pairs, unmatched tracks, unmatched detections.
7. Update matched tracks with their measurements.
8. Spawn new tracks from unmatched detections.
9. Coast unmatched tracks (increment `frames_since_seen`).
10. Delete stale tracks.
11. Select target via `select_target(tracks, pose)` (default: closest confirmed track with `hits` $\geq$ 3).
12. Publish target over UDP to RC.

2.10 Key Failure Modes to Design Against

- **Extrinsic miscalibration.** Produces consistent spatial offset between true and projected ball. Verify with a ball at a known location before trusting the pipeline.
- **Pose-frame time skew.** Produces an offset in the direction of robot motion. Enforce timestamp discipline.
- **Wheel-slip pose error.** Produces pose error during acceleration; filter absorbs some into spurious velocity. Use dead wheels where possible.
- **Isotropic measurement noise.** Makes far detections over-confident; causes track identity swaps. Always use the full R_{meas} from the Jacobian.
- **Stale target in control loop.** Always output track state predicted to *current* time, not last-update time, when handing to the chase controller.